Electronics and
Computer Science

UNIVERSITY OF
Southampton

5. Memory

Instruction and Data Memory

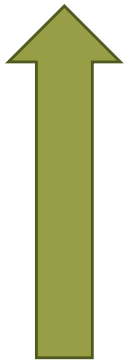
- Branch instructions modify the program counter to reference instruction memory
- Load and store instructions reference data memory
- In an ideal world, with infinite high-speed memory, we could have an ideal Harvard architecture
- In the modified Harvard model, L1 caches are separated, but higher levels share memory
- Before we look at subroutine calls (jump instructions), let's consider real memory

RISC-V Memory Space

- 32 bit addresses in register file for load and store
  - $2^{32} \sim 10^9 = 4GB$
- Memory is assumed to be 8-bits wide
  - Hence, Program Counter (PC) increments by 4 to get 32 bit instructions
  - Memory access is 4 addresses at a time
  - Load and store can reference half-words and bytes but as part of 4 byte access
- Memory space is therefore $2^{32} \div 4 = 2^{30} = 10^9$ addresses

Memory Hierarchy

- Registers
- SRAM
- DRAM
- Disk (flash)

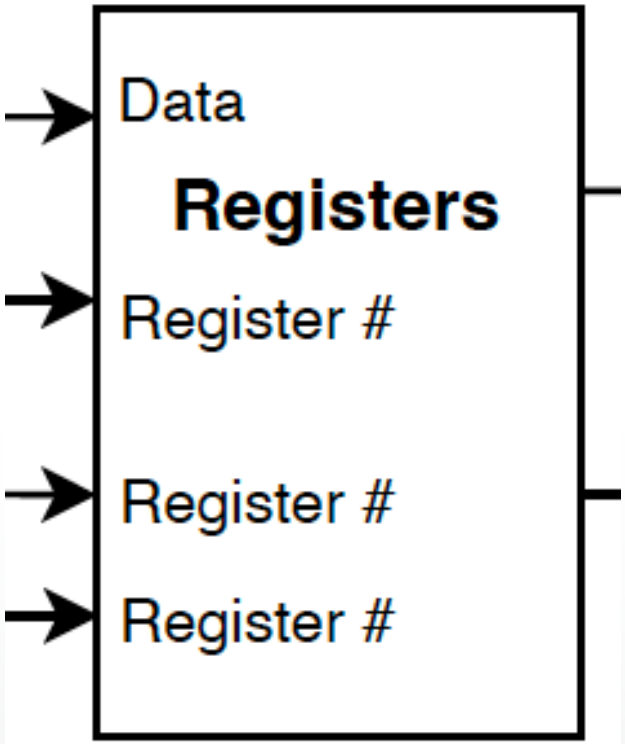


Increasing cost/bit
Increasing speed

Note that Static RAM (SRAM) is CMOS; Dynamic RAM (DRAM) is a different technology. Can't be integrated on the same chip.

Without on-chip caches, processors would stall because DRAM is slow. Fastest read time is ~20ns (50 MHz).

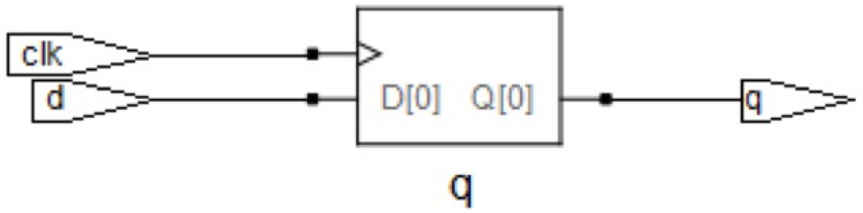
Register File



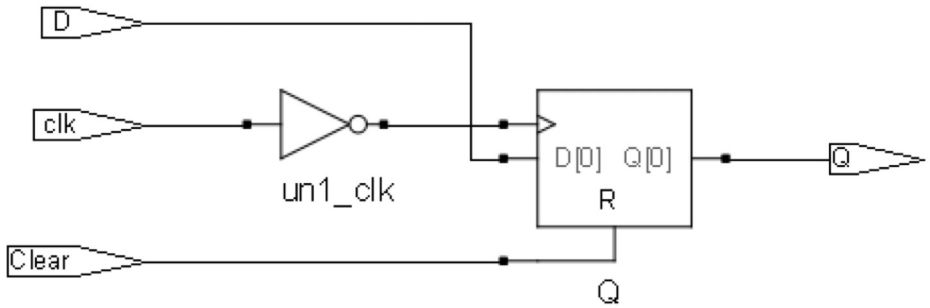
- One write address, rd
- 2 read addresses, rs1, rs2
- Address 0 always returns 0

Recap – Edge-triggered flip-flops

```
// rising edge triggered D-type flip-flop with no reset
module ff1 (input logic clk, d, output logic q);

always_ff @ (posedge clk)
     q <= d;

endmodule
```

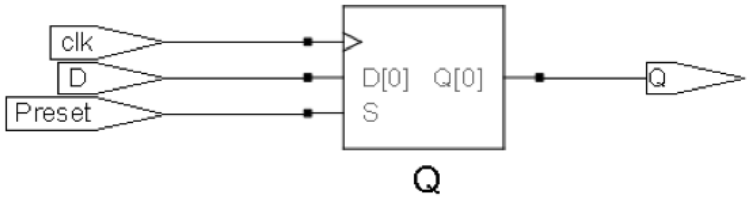



```
// falling edge triggered D-type flip-flop with
// active-high asynchronous reset
module ff2 (input logic clk, D, Reset,
            output logic  Q);

always_ff @ (negedge clk, posedge Reset)
     if (Reset)
       Q <= 1'b0;
    else       Q <= D;
endmodule
```

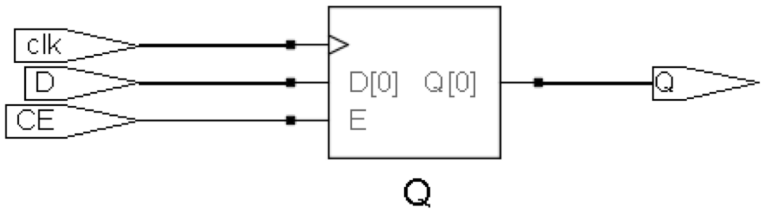


Edge-triggered flip-flops – cont.

```
// rising edge triggered D-type flip-flop with
// active-high synchronous preset
module ff3 (input logic clk, D, Preset, output
logic Q);

always_ff @ (posedge  clk)
   if (Preset)
        Q <= 1'b1;
      else
        Q <= D;

endmodule
```




```
// rising edge triggered D-type flip-flop with clock enable
module ff4 (input logic clk, D, CE, output logic Q);

always_ff @ (posedge clk)
     if (CE)
           Q <= D;

endmodule
```



RAM synthesis

- RAMs used in older FPGAs are typically arrays of latches with a separate input and a separate output data bus, an address bus and a Write Enable signal; read is asynchronous
- Modern FPGAs support only synchronous memory, where both read and write is synchronous
- Note that there many types of RAMs, eg.
  - RAM with synchronous read
  - RAM with one Enable controlling both ports
  - RAM with separate Enables controlling each port
  - Multiple-Port RAMs
- Synthesis tools are usually able to infer the correct type of RAM supported by the target FPGA, regardless of the SystemVerilog description, but it is useful to be familiar with the specific types of RAM supported by the target FPGA.

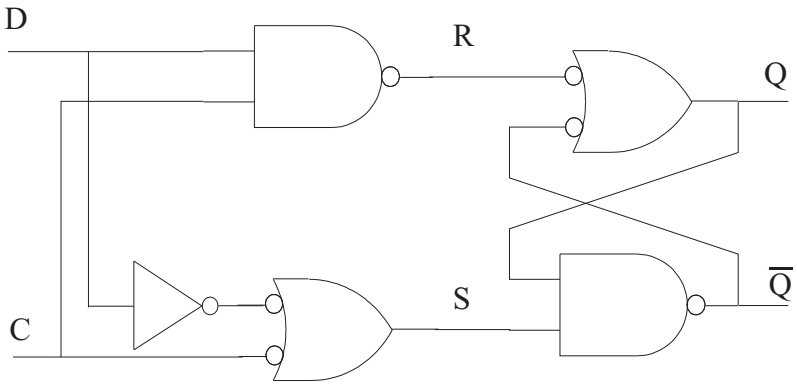
Three Port Register File

```
module registerfile
(input logic clk, writeBack, // clk and write control
 input logic [31:0] Wdata,
 input logic [4:0] rs1, rs2, rd,
 output logic [n-1:0] Rdata1, Rdata2);

    logic [31:0] rf [32]; // Declare 32 x 32-bit registers

    always_ff @ (posedge clk) // write process, dest reg is rd
      if (writeBack && rd !=0)
        rf[rd] <= Wdata;

    always_comb // read process, output 0 if %0 is selected
      begin
      if (rs1=='0)
        Rdata1 = '0;
      else Rdata1 = rf[rs1];
      if (rs2=='0)
        Rdata2 = '0;
      else Rdata2 = rf[rs2];
      end
endmodule
```

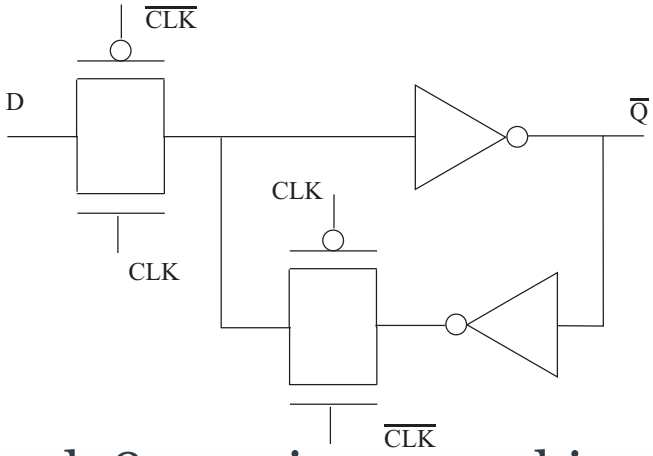
Register file implementation

- Each bit is a latch or flip-flop
  - In practice, use a "macro" on an FPGA – pre-optimised
- Relatively expensive (see next slide)
  - about 20 transistors per bit for an edge-triggered flip-flop (don't need set or reset)
- 31 x 32 registers (exclude address 0) = 992 bits, ~20k transistors + control logic
- Very fast, suitable for CPU registers

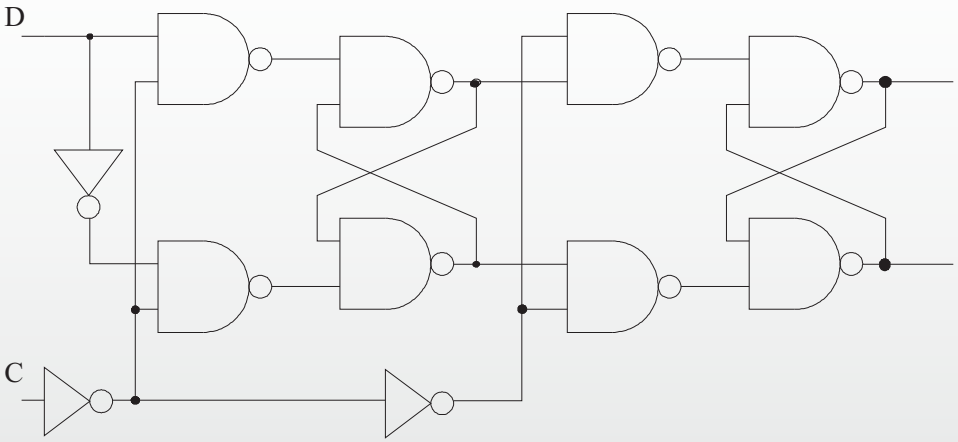
Static CMOS



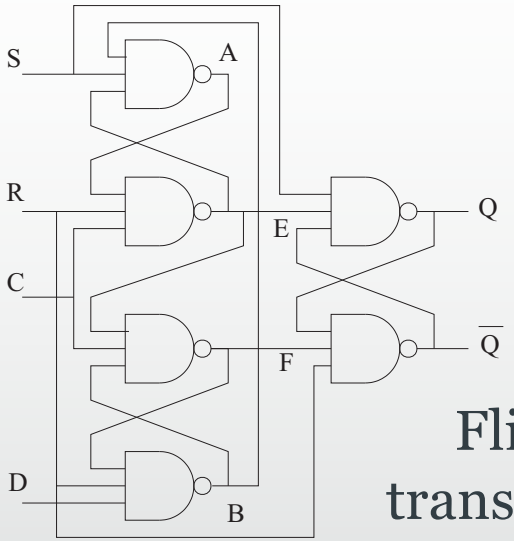
Latch 10 transistors per bit



Latch 8 transistors per bit

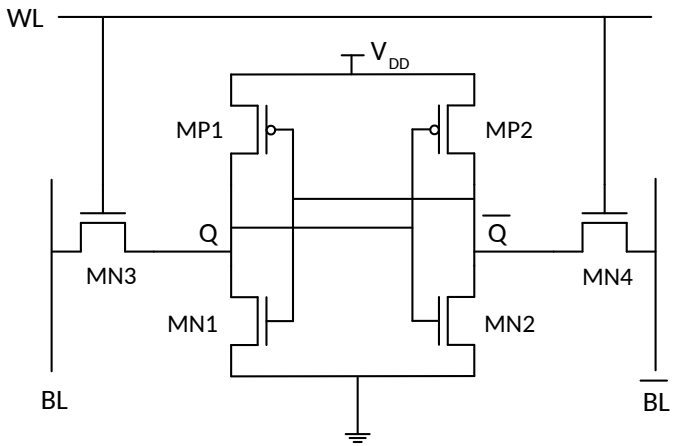


Flip-flop 18 transistors per bit



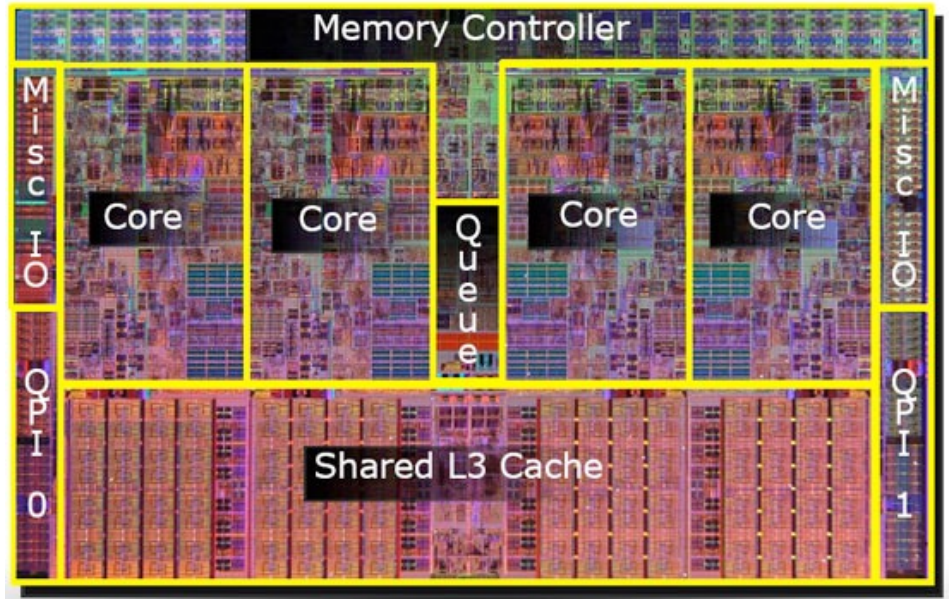
Flip-flop 36 transistors per bit

SRAM



- 6 Transistors per bit + control logic
- On-chip caches

On-chip cache



intel.com

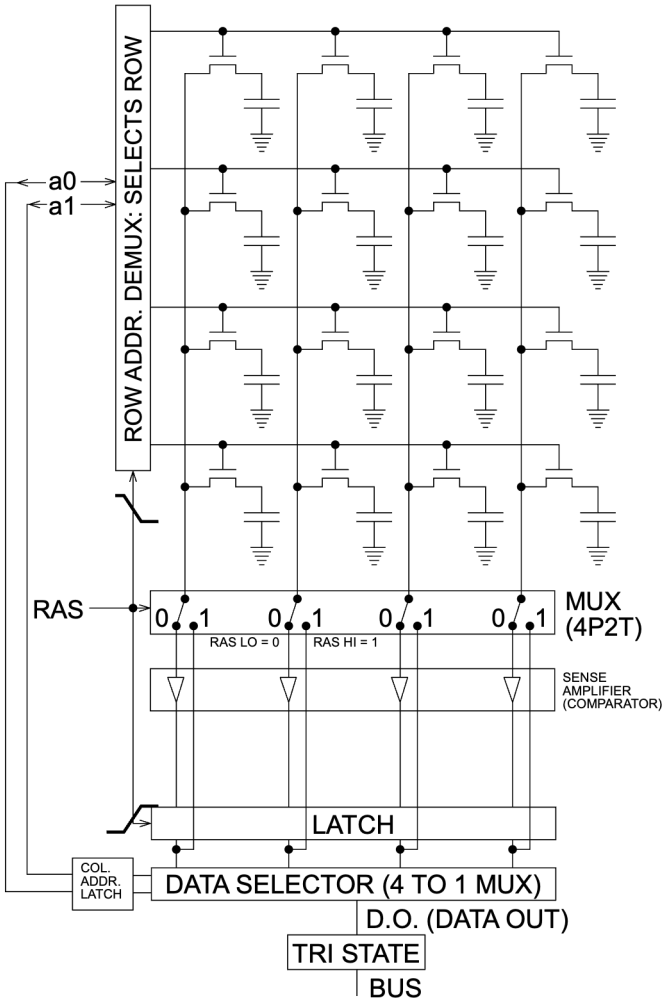
Intel i7

- On-chip SRAM is fast but too expensive to hold all instructions/data
- e.g. i7
  - 32k L1 I-cache
  - 32k L1 D-cache
  - 256k L2 cache
  - 8M L3 cache (shared)
- Nowhere near 4GB!

SystemVerilog Model of SRAM

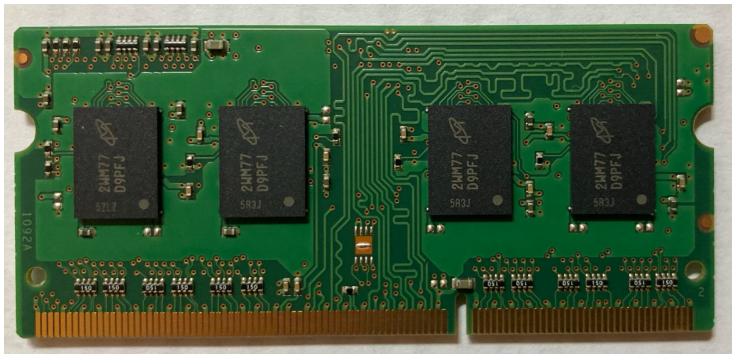
```
module RAM16x8(inout wire [7:0] Data,
               input logic [3:0]Address,
               input logic n_CS, n_WE, n_OE);

  logic [7:0] mem [16];

  assign Data = (~n_CS & ~n_OE) ? mem[Address] : 'z;

  always_latch //latch - no clock
    if (~n_CS & ~n_WE & n_OE)
      mem[Address] <= Data;

endmodule
```

DRAM



By Encheart at English Wikipedia (Derivative SVG), Glogger at English Wikipedia (Original Image) - File:Square_array_of_mosfet_cells_read.png, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=63506811

- 1 Transistor + 1 capacitor per bit
- Stored as charge
  - Needs to be refreshed periodically
- Very densely packed

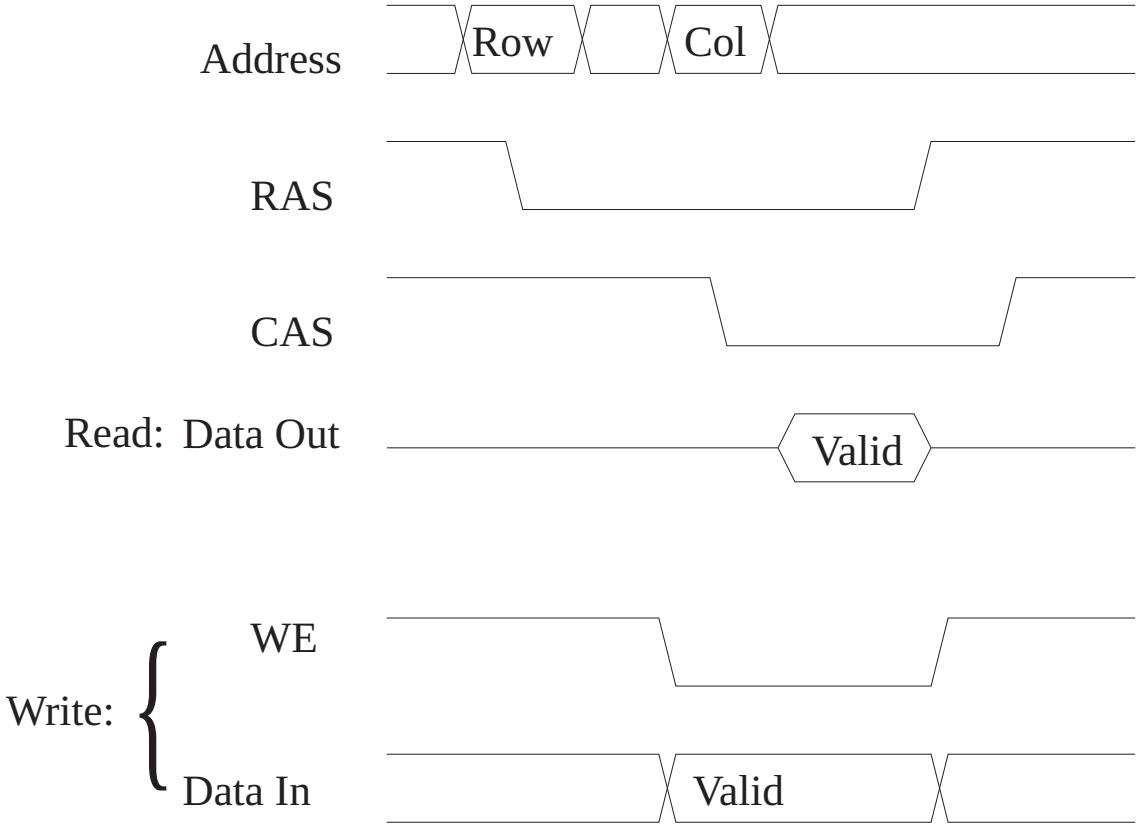


SystemVerilog Model of 8Mb DRAM

```
module DRAM8Mbit(inout wire [7:0] Data,
  input logic [9:0] Address, input logic RAS, CAS, WE);

  logic [7:0] mem [2**20];
  logic [9:0] row_address;
  logic [19:0] mem_address;

assign Data = (~CAS & ~RAS & WE) ? mem[mem_address] : 8'bz;

always @(negedge RAS)
  row_address <= Address;

always @(negedge CAS)
  begin
  mem_address = {row_address, Address};
  if (~RAS & ~WE)
    mem[mem_address] = Data;
  end
endmodule
```

Simulation only!
Don't try to synthesise!

DRAM Timing



Address is written in 2 phases: row, followed by column.
Timing is critical. This is a simplified timing diagram.

Control Logic

- The control logic for SRAM, DRAM not shown
- There needs to be interface logic to handle requests and manage timing
- DRAM needs refresh logic
- Caches need hardware
- etc., etc.
- We'll come back to some of this later

HDDs, SSDs

- After DRAM, there are Hard Disk Drives, Solid State Drives, etc.
  - All much, much slower than DRAM
- Programs, data can be stored on disk
- Running programs can use disks as Virtual Memory
  - Again, leave for another day

Exercise

- Write a SystemVerilog model of a parameterised SRAM
  - Parameterise number of bits, and number of addresses
  - Remember, 2^N is written as 2**N or 1<<N
  - $\log_2(16) = 4$; $\log_2(20) = 4.3$
  - Can't have 4.3 bits, so use "ceiling $\log_2$"
  - $clog2(20) = 5